

UNIVERSITÉ PARIS-DAUPHINE

PROJET DE DEEP LEARNING

---

# Génération de musique conditionnée par l'émotion

---

*Étude et implémentation d'un Auto-Encodeur Variationnel Conditionnel  
(CVAE-LSTM) sur le dataset EMOPIA*

Éric CHEN  
Léa YANG

Mai 2026

# Table des matières

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b><u>Introduction</u></b>                                        | <b>2</b>  |
| <b>2</b> | <b><u>Présentation et Préparation des Données</u></b>             | <b>2</b>  |
| 2.1      | <u>Le Jeu de Données EMOPIA</u>                                   | 2         |
| 2.1.1    | <u>Le Modèle Circomplexe de Russell (1980)</u>                    | 2         |
| 2.2      | <u>Tokenisation de nos données MIDI</u>                           | 3         |
| 2.3      | <u>Analyse Statistique et Séparabilité Initiale des Données</u>   | 3         |
| 2.4      | <u>La projection t-SNE</u>                                        | 4         |
| 2.5      | <u>L'Analyse en Composantes Principales (PCA)</u>                 | 5         |
| <b>3</b> | <b><u>Le VAE : Auto-Encodeur Variationnel</u></b>                 | <b>5</b>  |
| 3.1      | <u>L' Embedding</u>                                               | 5         |
| 3.2      | <u>L'architecture d'un VAE</u>                                    | 6         |
| 3.2.1    | <u>La Reparamétrisation</u>                                       | 6         |
| 3.2.2    | <u>Fonction de Loss : L'ELBO</u>                                  | 6         |
| 3.2.3    | <u>Résultats Attendus</u>                                         | 7         |
| 3.3      | <u>Protocole d'Entraînement et Stratégies de Régularisation</u>   | 7         |
| 3.4      | <u>Test 1 : Approche de Base (VAE avec MLP)</u>                   | 7         |
| 3.5      | <u>Test 2 : Approche Spatiale (VAE avec CNN)</u>                  | 9         |
| 3.6      | <u>Test 3 : La Solution Séquentielle (VAE avec LSTM)</u>          | 10        |
| 3.6.1    | <u>Qu'est-ce qu'un LSTM ?</u>                                     | 10        |
| <b>4</b> | <b><u>Le C-VAE : Auto-Encodeur Variationnel conditionnel</u></b>  | <b>11</b> |
| 4.1      | <u>L'architecture d'un C-VAE</u>                                  | 12        |
| <b>5</b> | <b><u>Génération et Résultats :</u></b>                           | <b>12</b> |
| 5.1      | <u>Analyse de la Convergence et de la Séparation Émotionnelle</u> | 13        |
| <b>6</b> | <b><u>Conclusion et Perspectives</u></b>                          | <b>15</b> |
| 6.1      | <u>Bilan des Modèles</u>                                          | 15        |

# Génération de musique conditionnée par l'émotion

## 1 Introduction

La musique est fondamentalement un langage émotionnel. Avant même d'en analyser la structure harmonique, rythmique ou mélodique, l'auditeur perçoit une intention, une ambiance, un sentiment. Cependant, l'émotion musicale est par nature profondément subjective. Elle ne dépend pas uniquement des ondes sonores, mais aussi de la culture de l'auditeur, de son éducation musicale et de son état psychologique au moment de l'écoute. Une mélodie perçue comme mélancolique par un individu peut très bien être ressentie comme apaisante par un autre.

Cette subjectivité inhérente à la musique pose un défi colossal pour l'intelligence artificielle. La plupart des modèles génératifs produisent une musique qui, bien que techniquement correcte, manque de direction et d'intention. Le résultat sonne souvent de façon aléatoire, incapable de transmettre une véritable émotion humaine.

La motivation principale de ce projet est de générer une musique qui suit une émotion particulière. Notre objectif n'est pas de créer une IA qui génère des notes au hasard, mais de développer un système capable de composer "sur commande", en répondant à une directive émotionnelle précise (la joie, la colère, la tristesse, le calme).

Pour enseigner cette subjectivité à une machine, il faut d'abord traduire l'émotion dans un cadre mesurable. C'est pourquoi nous utilisons la base de données EMOPIA qui associe à chaque morceau de piano une étiquette en fonction du modèle de Russell.

Pour atteindre cet objectif, nous avons d'abord essayé de trouver le VAE le plus efficace en génération de morceau en essayant plusieurs modèles différents puis nous avons utilisé un Auto-Encodeur Variationnel Conditionnel (C-VAE), couplé à des réseaux capables de comprendre les séquences (LSTM) pour pouvoir générer des sons propres à chaque émotion.

## 2 Présentation et Préparation des Données

### 2.1 Le Jeu de Données EMOPIA

Le projet repose sur l'utilisation du dataset EMOPIA (Emotion in Piano Performance). Il s'agit d'une base de données de morceaux de piano seul, annotés selon le modèle circomplexe des émotions de Russell.

#### 2.1.1 Le Modèle Circomplexe de Russell (1980)

Développé par le psychologue James Russell, ce modèle propose de ne pas voir les émotions comme des boîtes isolées (joie, tristesse, peur), mais comme un espace continu en deux dimensions.

Toute émotion humaine peut être placée sur un graphique grâce à deux axes :

- L'axe horizontal (La Valence) : Il mesure la nature de l'émotion, allant du négatif/désagréable (à gauche) au positif/agréable (à droite).
- L'axe vertical (L'Activation ou Arousal) : Il mesure l'intensité ou l'énergie physique de l'émotion, allant de faible/calme (en bas) à forte/excitée (en haut).

En croisant ces deux axes, on obtient les fameux 4 quadrants (4Q) utilisés dans EMOPIA (qui constitue les labels de nos données) :

L'analyse des fichiers MIDI a permis de confirmer que le placement dans les 4 quadrants de Russell est profondément lié aux caractéristiques musicales fondamentales des morceaux<sup>1</sup> :

- Le lien avec l'Activation (Arousal) : L'intensité de l'émotion se traduit directement par l'énergie du jeu. L'analyse révèle que les quadrants à haute activation (Q1 et Q2) se distinguent par une forte densité de notes (beaucoup de notes jouées par battement) et une vitesse élevée (une frappe forte sur les touches du piano). À l'inverse, les quadrants à basse activation (Q3 et Q4) se caractérisent par des notes tenues plus longtemps et jouées plus doucement.
- Le lien avec la Valence : La "positivité" de l'émotion est intimement liée à la tonalité harmonique. La distribution des tonalités montre que les morceaux à haute valence (Q1 et Q4) utilisent très majoritairement des gammes majeures. En revanche, les morceaux à basse valence (Q2 et Q3), qui expriment la tension ou la tristesse, s'appuient massivement sur des gammes mineures.

| Quadrant  | Valence | Activation (Arousal) | Émotions Associées                |
|-----------|---------|----------------------|-----------------------------------|
| <b>Q1</b> | Haute   | Haute                | Joie, excitation, triomphe        |
| <b>Q2</b> | Basse   | Haute                | Colère, tension, anxiété          |
| <b>Q3</b> | Basse   | Basse                | Tristesse, dépression, mélancolie |
| <b>Q4</b> | Haute   | Basse                | Calme, tendresse, relaxation      |

TABLE 1 – Les quatre quadrants du modèle circomplexe de Russell.

## 2.2 Tokenisation de nos données MIDI

Nous avons donc une base de données composée de 1078 séquences de format MIDI. Les fichiers audio MIDI permettent de représenter exactement quand une note est pressée et pour combien de temps, ce qui évite les bruits parasites. Les fichiers MIDI bruts ne sont pas directement exploitables par des réseaux de neurones. Pour modéliser la musique avec des algorithmes issus du Traitement du Langage Naturel (NLP), il est nécessaire de tokeniser nos données. Dans ce projet, nous avons utilisé la librairie MidiTok<sup>2</sup> pour convertir les fichiers MIDI en séquences de "mots" musicaux (tokens) en utilisant la représentation REMI (REvamped MIDI).<sup>3</sup>

Cette méthode consiste à convertir une séquence d'événements musicaux continus en une suite de variables catégorielles discrètes (tokens). REMI structure la musique sur une grille métrique (mesures et pulsations), ce qui permet de modéliser le tempo et la rythmique de chaque morceau. Une note n'est pas un événement isolé, mais est toujours contextualisée par sa position dans la mesure. Un morceau de musique est alors transformé en une séquence ordonnée de tokens composés de plusieurs éléments :

1. Bar : Indique le début d'une nouvelle mesure.
2. Position : Divise la mesure en une grille (ex : 16 ou 32 subdivisions). Cela indique quand la note est jouée.
3. Pitch : La hauteur de la note (0 à 127 en MIDI).
4. Velocity : La force de frappe (quantifiée ici sur 16 niveaux pour simplifier l'apprentissage).
5. Duration : La durée de la note (exprimée en fractions de pulsation).

Par exemple, pour une note jouée, on se retrouve avec une séquence : [Bar\_0] → [Position\_0] → [Pitch\_60] → [Velocity\_12] → [Duration\_2.0] → [Position\_8] → ...

La taille de notre vocabulaire final (le nombre d'événements musicaux uniques) s'élève à 268 tokens uniques.

## 2.3 Analyse Statistique et Séparabilité Initiale des Données

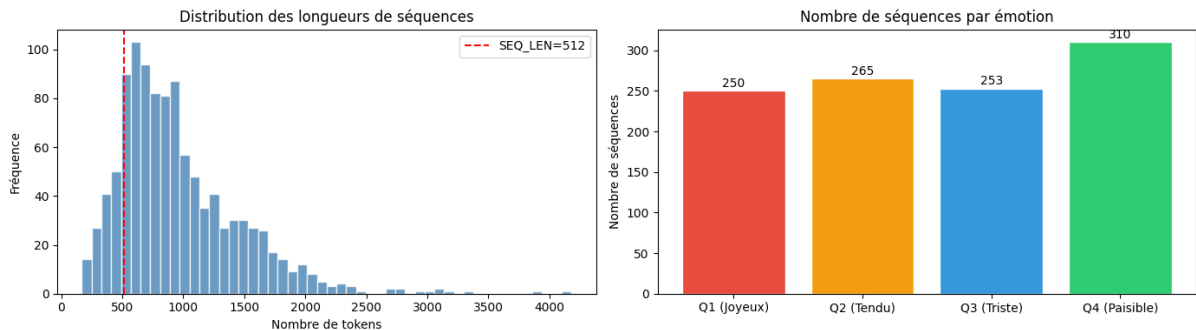


FIGURE 1 – graphiques descriptifs de la base de donnée

Afin de mieux comprendre la structure du jeu de données EMOPIA avant d'entraîner notre modèle, nous avons procédé à une analyse des séquences tokenisées.

Comme l'illustre le diagramme en barres (graphique de droite), le dataset présente une distribution relativement équilibrée entre les quatre émotions, allant de 250 échantillons pour la classe Q1 (Joyeux) à 310 pour la classe Q4 (Paisible). Cet équilibre garantit que le réseau de neurones ne développera pas de biais statistique majeur en faveur d'une émotion dominante lors de l'apprentissage.

Concernant la longueur des morceaux, l'histogramme des distributions révèle que la majorité des séquences originales contiennent entre 400 et 1500 tokens. Les réseaux de neurones (et particulièrement l'entraînement par batches) exigent des dimensions fixes.

Nous avons fait le choix d'imposer une longueur fixe par troncature à nos données ce qui implique qu'une partie du développement musical de fin de morceau sera sacrifiée. Nous avons fixé une longueur maximale de  $\text{SEQ\_LEN} = 512$  tokens (déterminée après plusieurs tests et en suivant le graphique ci-dessus). Ce choix est un compromis : il est suffisamment long pour conserver une part significative du développement musical original (contrairement à une coupe trop courte), tout en standardisant les entrées. Le graphique montre clairement un "plafond" à 500-600 atteint par la majorité des séquences, tandis que les nombreux points situés en dessous représentent les morceaux naturellement plus courts, qui ont été uniformisés à l'aide de padding qui permet de remplir par 0 les données trop courtes.

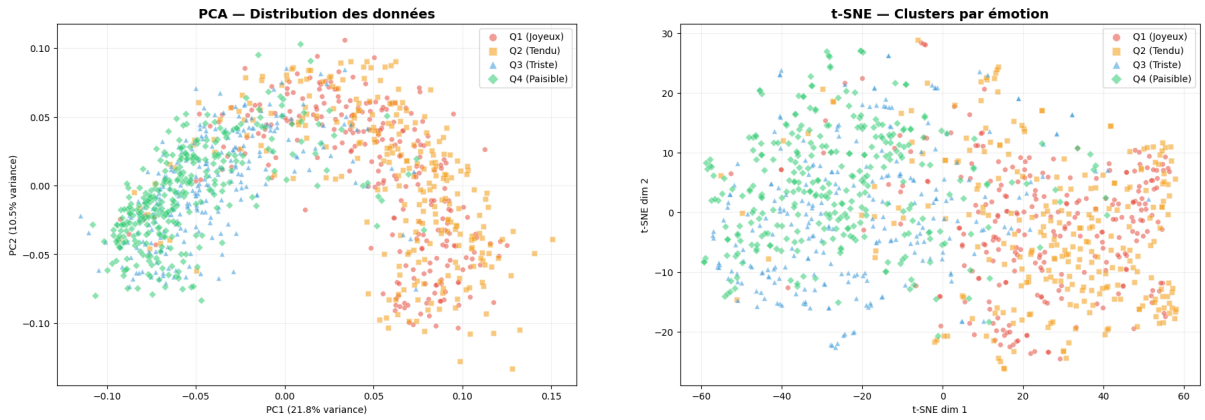


FIGURE 2 – graphique des espaces latents des données initiales

## 2.4 La projection t-SNE

Pour visualiser nos données en grande dimension, nous utilisons le t-SNE.<sup>4</sup> Le t-SNE cherche à préserver la topologie locale : des séquences musicales proches dans l'espace multidimensionnel d'origine resteront voisines sur le graphique 2D.

L'algorithme du t-SNE convertit les distances euclidiennes entre les points de données en probabilités conditionnelles. La probabilité  $p_{j|i}$  que le point  $x_i$  choisisse le point  $x_j$  comme voisin est proportionnelle à la densité d'une loi Gaussienne centrée sur  $x_i$  :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

Ainsi, deux séquences musicales très similaires auront une probabilité  $p$  proche de 1, tandis que deux séquences très différentes auront une probabilité proche de 0. Les points sont ensuite placés aléatoirement sur un plan 2D (les points  $y_i$ ). Pour mesurer leur similarité dans ce nouvel espace, l'algorithme utilise une loi de Student à un degré de liberté (loi de Cauchy) :

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}}$$

L'utilisation de la loi de Student permet de repousser les points modérément éloignés beaucoup plus loin sur la carte 2D.

L'objectif du t-SNE est de rendre la distribution des voisinages en 2D ( $q$ ) la plus fidèle possible à la distribution d'origine en haute dimension ( $p$ ). L'algorithme minimise donc la Divergence de Kullback-

Leibler (KL) entre ces deux distributions à l'aide d'une descente de gradient :

$$C = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

### Interprétation de notre graphique t-SNE :

Pour vérifier si les émotions se distinguaient facilement par une simple analyse de la fréquence des notes jouées, nous avons projeté les données brutes en deux dimensions. La projection t-SNE montre un chevauchement des quatre quadrants. Aucune frontière claire ne sépare naturellement la joie, la tristesse, la sérénité et la tension. Dans notre analyse future, nous utiliserons le t-SNE pour juger si notre modèle est efficace. S'il parvient à créer des clusters distinctes sur notre graphique 2D selon les couleurs de nos émotions, cela prouvera mathématiquement que notre modèle de Deep Learning a réussi à rapprocher les caractéristiques structurelles des morceaux partageant une même intention émotionnelle.

## 2.5 L'Analyse en Composantes Principales (PCA)

La PCA (Principal Component Analysis)<sup>5</sup> est également une technique de réduction de dimensionnalité. Contrairement au t-SNE (qui est non-linéaire et focalisé sur les voisinages locaux), la PCA est une méthode linéaire et globale. Son objectif est de trouver les angles d'observation qui permettent de capturer le maximum de variance de nos données d'origine. Dans notre cas, chaque morceau de musique

est représenté par la fréquence d'apparition de ses 268 tokens bruts (un espace à 268 dimensions). L'algorithme analyse d'abord comment les 268 tokens varient les uns par rapport aux autres à travers tout le jeu de données (par exemple, si le token d'un accord mineur apparaît souvent en même temps qu'un tempo lent). Puis il calcule les vecteurs propres de cette matrice. La Valeur Propre associée indique la quantité de variance (d'information) que cette ce vecteur propre contient. Puis il sélectionne les composantes principales : La première composante principale (PC1, l'axe horizontal) représente l'axe où nos

données sont le plus étalées ; elle capture à elle seule 21,8 % de la diversité (la variance) de notre jeu de données. La deuxième composante (PC2, l'axe vertical), tracée perpendiculairement, en capture 10,5 %. Ainsi, en projetant nos 1 078 morceaux de musique sur ce simple graphique en 2D, nous parvenons à conserver et visualiser près d'un tiers (32,3 %) de l'information d'origine. Visuellement, la distribution de

nos points forme une courbure caractéristique en arc de cercle. En statistiques, ce phénomène est l'Effet Guttman . Cet arc apparaît lorsque l'on force un algorithme linéaire à dessiner des données qui suivent une progression fluide. Pour notre projet, cette forme en arc prouve que nos morceaux n'évoluent pas par paliers brutaux, mais de manière très progressive. Toutefois, en observant le graphique, on remarque que les quatre classes représentant nos émotions (Joie, Tristesse, Colère, Calme) se mélangent et se superposent fortement le long de cet arc. Ce chevauchement massif prouve qu'il est impossible de tracer une simple ligne droite pour séparer parfaitement une émotion d'une autre (c'est ce qu'on appelle l'absence de "séparabilité linéaire").

C'est précisément cette difficulté qui justifie tout l'intérêt de notre projet. Puisqu'une méthode mathématique simple (l'ACP) échoue à isoler les émotions, nous avons la preuve scientifique qu'il faut recourir à l'apprentissage profond (Deep Learning). L'utilisation d'une architecture complexe et non linéaire comme l'Auto-Encodeur Variationnel Conditionnel (C-VAE) devient alors indispensable pour réussir à forcer cette séparation et apprendre à l'ordinateur à distinguer clairement la Joie de la Tristesse.

## 3 Le VAE : Auto-Encodeur Variationnel

Dans un premier temps, nous avons choisi d'utiliser les architectures de type Auto-Encodeur Variationnel (VAE) pour trouver le modèle le plus efficace dans la génération de morceaux de piano, avant de nous diriger vers une approche conditionnelle.

### 3.1 L' Embedding

Afin que nos modèles d'apprentissage profond puissent analyser la musique, les séquences de tokens doivent être projetées dans un espace mathématique continu. Quelle que soit l'architecture du réseau de neurones utilisée par la suite (Perceptron Multicouche, CNN ou LSTM), la toute première étape consiste à traverser une couche d'Embedding (`nn.Embedding`).

L'Embedding associe chaque token à un vecteur dense de taille fixe (ex : une dimension de 128). Cette couche agit comme une matrice de correspondance  $W_E \in \mathbb{R}^{|V| \times d}$ , où  $|V|$  représente la taille totale de notre vocabulaire (environ 268 tokens uniques) et  $d$  la dimension du plongement (128). Ainsi, un token discret est transformé en un vecteur continu en sélectionnant simplement la ligne correspondante dans cette matrice.

L'intérêt de cette approche réside dans sa capacité d'apprentissage. Les poids de ces vecteurs sont initialisés de manière aléatoire, mais elles sont continuellement ajustées par backpropagation tout au long de l'entraînement. Le modèle apprend donc progressivement à sculpter cet espace géométrique pour y regrouper les tokens partageant des fonctions musicales similaires.

### 3.2 L'architecture d'un VAE

L'Auto-Encodeur Variationnel (VAE) apprend à compresser les données dans un espace latent de faible dimension, puis à les reconstruire. Contrairement à un auto-encodeur classique, le VAE impose une structure probabiliste stricte sur l'espace latent.

L'utilisation d'une architecture de type Auto-Encodeur pour la musique est d'ailleurs fortement justifiée par l'état de l'art en traitement du signal musical.<sup>6</sup> En effet, la musique (modélisée en séquences symboliques) est une donnée de très haute complexité temporelle. Un Auto-Encodeur permet de forcer le réseau à extraire les corrélations profondes et globales des morceaux (tonalité, rythme, expressivité) en les réduisant dans un espace latent structuré, condition indispensable pour pouvoir générer ensuite des compositions musicales fluides et cohérentes.

Le processus complet du VAE peut être modélisé ainsi :

$$x \text{ (tokens MIDI)} \rightarrow \text{Encodeur} \rightarrow \mu, \log(\sigma^2) \rightarrow z \sim \mathcal{N}(\mu, \sigma^2) \rightarrow \text{Décodeur} \rightarrow \hat{x} \text{ (reconstruction)}$$

#### 3.2.1 La Reparamétrisation

L'échantillonnage aléatoire classique  $z \sim \mathcal{N}(\mu, \sigma^2)$  empêche la rétropropagation du gradient. L'astuce de reparamétrisation résout ce blocage : au lieu d'échantillonner directement  $z$ , on calcule :

$$z = \mu + \sigma \cdot \epsilon \quad \text{où} \quad \epsilon \sim \mathcal{N}(0, 1)$$

Ici,  $\epsilon$  reste fixe pendant la passe retour (*backward pass*). Ainsi, le gradient mathématique peut passer par  $\mu$  et  $\sigma$  pour mettre à jour les poids du réseau, sans être interrompu par l'échantillonnage.

#### 3.2.2 Fonction de Loss : L'ELBO

Le VAE maximise mathématiquement l'ELBO (*Evidence Lower Bound*) :

$$\text{ELBO} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x) || p(z))$$

En pratique lors de l'entraînement, on cherche à minimiser la fonction de perte  $\text{Loss} = -\text{ELBO} = \text{Reconstruction} + \text{KL}$ .

**1. Loss de Reconstruction :** Elle mesure à quel point le décodeur reconstruit bien les tokens originaux. On utilise la Cross-Entropy (le standard absolu pour la classification multi-classes) :

$$\mathcal{L}_{recon} = - \sum_{t=1}^T \log p(x_t|z)$$

Il est important de noter que le padding (le token 0 ajouté pour uniformiser les séquences) est ignoré lors de ce calcul grâce au paramètre `ignore_index=0`, afin de ne pas biaiser la perte.

**2. KL Divergence :** Ce terme force l'espace latent à rester proche d'une Gaussienne standard  $\mathcal{N}(0, 1)$ . Pour deux Gaussiennes, la divergence KL a une forme analytique exacte :

$$\mathcal{L}_{KL} = \text{KL}(q || p) = -\frac{1}{2} \sum_{j=1}^d (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2)$$

**KL Annealing ( $\beta$ -warmup) :** Au début de l'entraînement, un modèle a naturellement tendance à ignorer totalement l'espace latent (phénomène de *KL collapse*). Pour pallier ce problème, la perte KL est pondérée par un facteur  $\beta$  qui augmente très progressivement de 0 à 1 (sur plusieurs époques). Cela force doucement le modèle à structurer son espace latent une fois qu'il a commencé à apprendre à reconstruire la musique.

### 3.2.3 Résultats Attendus

Avec cette configuration, l'entraînement réussi d'un modèle doit présenter trois dynamiques :

1. La perte de reconstruction doit diminuer fortement (le modèle apprend à lire et reconstruire la partition).
2. La divergence KL doit augmenter puis se stabiliser (l'espace latent s'organise et devient utile).
3. La perte totale doit converger de manière symétrique entre l'entraînement et la validation, prouvant l'absence de surapprentissage.

## 3.3 Protocole d'Entraînement et Stratégies de Régularisation

**Séparation des données :** Les données sont divisées en deux ensembles distincts : un ensemble d'entraînement, utilisé pour apprendre les paramètres du modèle, et un ensemble de validation, utilisé pour évaluer sa capacité de généralisation. Cette séparation permet de vérifier que le modèle ne se contente pas de mémoriser les exemples d'entraînement, mais qu'il est capable de fonctionner correctement sur des données qu'il n'a jamais vues.

**Principe d'apprentissage :** Le modèle apprend de manière itérative à reconstruire des séquences musicales à partir de leurs représentations internes. Elle a pour objectif de minimiser la fonction de Loss ELBO. Au fil de l'entraînement, il construit progressivement une représentation compacte des données (espace latent), qui permet de résumer l'information musicale essentielle tout en facilitant la génération.

**Early stopping :** La performance du modèle est régulièrement évaluée sur l'ensemble de validation afin de suivre son évolution. Lorsque cette performance s'améliore, le modèle est sauvegardé comme meilleure version actuelle. En revanche, si aucune amélioration n'est observée pendant un certain nombre d'étapes consécutives, l'entraînement est arrêté. Cette stratégie permet d'éviter le surapprentissage et de conserver un modèle qui généralise mieux aux données nouvelles.

**Modèle final :** Le modèle final correspond à celui qui a obtenu les meilleures performances sur l'ensemble de validation au cours de l'entraînement. Il représente ainsi le meilleur compromis entre la qualité de reconstruction des séquences musicales et la capacité de généralisation du modèle.

## 3.4 Test 1 : Approche de Base (VAE avec MLP)

Pour initier notre phase expérimentale, nous avons implémenté une architecture VAE s'appuyant sur des réseaux de neurones linéaires (Perceptrons Multicouches ou MLP). L'objectif premier était d'établir une baseline (modèle de référence) simple et rapide à entraîner. Ce choix méthodologique nous a permis de valider l'intégralité de notre pipeline de données — du traitement du dataset EMOPIA jusqu'à la conversion des tokens générés en fichiers MIDI — avant de déployer des architectures plus complexes et coûteuses en temps de calcul.

- **Configuration :** L'encodeur projette les 256 tokens (séquence imposée) via l'embedding de dimension 64, aplatit le résultat (vecteur de dimension 16 384) et le compresse à travers deux couches linéaires (512 puis 256 neurones) pour estimer  $\mu$  et  $\log(\sigma^2)$  dans un espace latent de dimension 32. Le décodeur reçoit ce vecteur de la dimension de l'espace latent, l'élargit de manière symétrique (dimensions 256 puis 512) et le projette vers une couche finale ( $256 \times 268$ ), remodelée pour restituer les logits sur le vocabulaire de 268 jetons. Le modèle comptait plus de 87 millions de paramètres.
- **Génération :** L'étape de génération s'effectue en échantillonnant directement le vecteur  $z$  dans l'espace latent. Les logits prédits par le décodeur sont convertis en tokens selon deux modes, contrôlés par un paramètre de température :
  - Mode Déterministe (Température  $\leq 0$ ) : Sélection stricte du token le plus probable ( $\text{argmax}$ ).
  - Mode Stochastique (Température  $> 0$ ) : Au lieu de choisir systématiquement la note la plus probable à l'issue du Softmax, le modèle tire au sort la note suivante parmi plusieurs options probables. En réglant la température, on ajuste ce tirage : une température basse rend le modèle très prudent et répétitif, tandis qu'une température élevée l'autorise à prendre des risques et à choisir des notes plus surprenantes. C'est ce mécanisme qui apporte de la variété et évite que les musiques ne se ressemblent toutes.

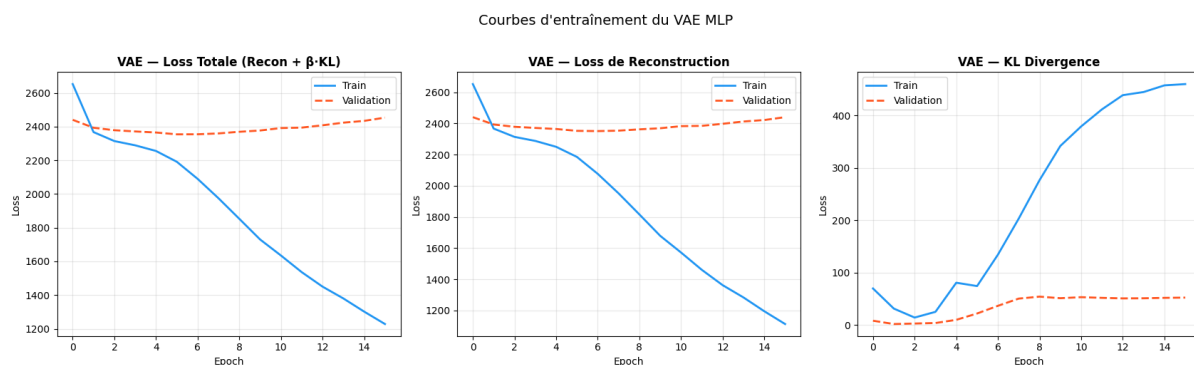


FIGURE 3 – graphique des loss du VAE MLP

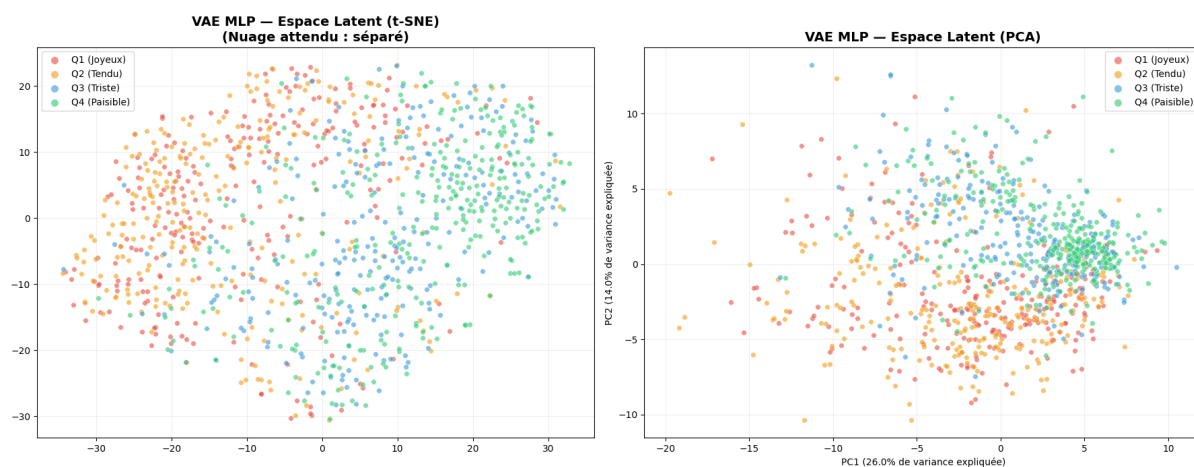


FIGURE 4 – graphique de l'espace latent du VAE MLP

### — Résultats et Problèmes :

L'apprentissage a immédiatement souffert d'un surapprentissage massif (la perte d'entraînement chutait à 1200 tandis que la validation stagnait à 2500). Le problème majeur réside dans l'explosion de la KL Divergence sur l'entraînement, indiquant que l'encodeur "triche" en éparpillant les données d'entraînement dans l'espace latent pour maximiser la mémorisation au détriment de la généralisation. Pour ce qui est de l'espace latent, les projections t-SNE et PCA révèlent un échec de séparation sémantique : les classes se chevauchent sans former de clusters distincts. Bien que l'espace paraisse désordonné, on peut observer une séparation binaire : le modèle parvient à séparer les émotions à forte activation (Q1/Q2) de celles à faible énergie (Q3/Q4). Cela montre que l'encodeur a correctement identifié la dimension de l'arousal (l'intensité), mais reste incapable de discriminer la valence (le caractère positif ou négatif), fusionnant ainsi la joie avec la tension et la tristesse avec l'apaisement.

- **Analyse Musicale** : Le résultat sonore était peu satisfaisant, beaucoup trop court, et composé de notes aléatoires. Le problème fondamental du MLP est qu'il traite chaque position de la séquence indépendamment.
- **Limite** : Absence de conscience temporelle. En aplatissant la séquence musicale pour l'injecter dans le MLP, le modèle détruit l'ordre chronologique des notes. Comme il ne possède aucun mécanisme de mémoire (récurrence) ou de reconnaissance de paterne, il est incapable de comprendre la suite logique des jetons. Cela élimine toute notion de rythme, de mélodie continue ou d'harmonie cohérente.

### 3.5 Test 2 : Approche Spatiale (VAE avec CNN)

L'échec de la baseline MLP-VAE à capturer la structure musicale met en évidence une nécessité fondamentale : pour traiter efficacement le dataset EMOPIA, le modèle ne doit pas simplement voir la musique comme un vecteur de données aplati, mais doit être capable de saisir les relations locales et les motifs répétitifs (motifs rythmiques, accords, progressions mélodiques). Pour pallier les deux défauts majeurs du MLP (l'absence de conscience temporelle et l'explosion du nombre de paramètres), nous avons choisi de nous tourner vers des réseaux de neurones convolutifs (CNN) appliqués en une dimension (CNN 1D)

- **Configuration** : L'encodeur possède 3 couches de convolution 1D, interposées par des fonctions d'activation ReLU et des couches de Dropout permettant d'éviter le surapprentissage. Le décodeur fonctionne selon le principe inverse : il reçoit le vecteur latent, le projette et le remodele pour appliquer des convolutions transposées 1D (ou déconvolutions) afin de reconstruire pas à pas la structure spatiale et temporelle de la séquence musicale d'origine jusqu'à restituer les logits. Le nombre de paramètres a été drastiquement réduit à environ 539 000.

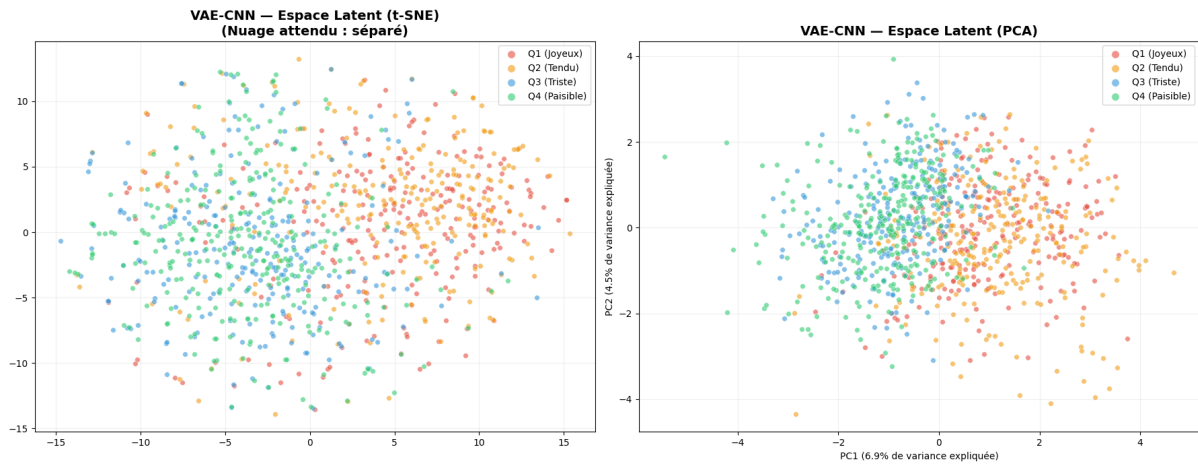


FIGURE 5 – graphique des loss du VAE CNN

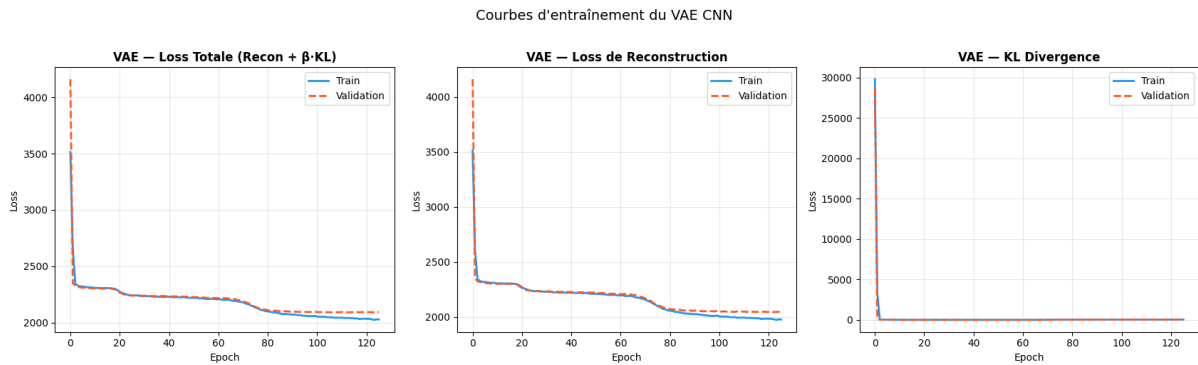


FIGURE 6 – graphique de l'espace latent du VAE CNN

- **Résultats** : Contrairement au MLP, les courbes de train et de validation sont quasi confondues. Il n'y a plus de surapprentissage massif ; le modèle généralise bien mieux ce qu'il apprend. La perte de reconstruction diminue (jusqu'à environ 1710), ce qui suggère que les couches convolutionnelles captent progressivement des motifs structurels. Cependant le graphique de la KL Divergence montre une chute brutale (atteint près de 116), c'est le signe d'un "effondrement" : la contrainte de régularisation est trop forte ou le décodeur CNN est trop puissant. Le modèle ignore l'espace latent et se comporte comme un auto-encodeur simple. Pour ce qui est de l'espace latent, nous n'avons toujours pas de distinction claire entre les différents groupes.

- **Analyse Musicale et Limites** : Malgré la réduction de la perte totale, la musique générée était hachée et pleine de silences. Le CNN génère l'ensemble des 512 tokens en parallèle. Il peine à respecter la grammaire stricte du tokenizer REMI (Position → Pitch → Velocity → Duration). De plus, les CNN évaluent la musique par petites fenêtres glissantes et sont frappés d'amnésie à court terme, les rendant incapables de maintenir une cohérence mélodique sur 15 secondes. Une partition n'est pas une image figée, mais un ensemble dynamique doté d'une cohérence et d'un enchaînement logique du début à la fin. Les filtres fixes du CNN peinent à capturer cela.

### 3.6 Test 3 : La Solution Séquentielle (VAE avec LSTM)

Pour résoudre le problème d'amnésie et de parallélisation, nous avons implémenté des Réseaux de Neurones Récurents, et plus spécifiquement un VAE-LSTM (Long Short-Term Memory).

#### 3.6.1 Qu'est-ce qu'un LSTM ?

Le *Long Short-Term Memory* (LSTM) est une architecture de réseau de neurones récurrent conçue pour traiter des données séquentielles tout en résolvant le problème de la disparition du gradient (*vanishing gradient*). Là où les gradients rétropropagés des RNN classiques tendent à s'atténuer de manière exponentielle, le LSTM introduit une cellule mémoire spécifique. Celle-ci repose sur la coexistence de deux états distincts :

- L'état caché ( $h_t$ ), qui représente la mémoire à court terme ;
- L'état de cellule ( $c_t$ ), un canal central à long terme qui permet à l'information de traverser toute la séquence temporelle sans altération.

Pour réguler ce flux d'informations, le LSTM utilise trois portes mathématiques qui s'activent à chaque étape temporelle  $t$  :

1. **La porte d'oubli** (*forget gate*) : elle élimine les données antérieures de l'état  $c_{t-1}$  devenues obsolètes.
2. **La porte d'entrée** (*input gate*) : elle sélectionne les éléments pertinents du jeton actuel  $x_t$  pour mettre à jour la mémoire à long terme  $c_t$ .
3. **La porte de sortie** (*output gate*) : elle filtre cette mémoire consolidée afin de générer le nouvel état caché  $h_t$ .

Ce mécanisme de contrôle permet au réseau de décider dynamiquement ce qu'il doit retenir ou ignorer au fil de la séquence.

### Application à la génération musicale

Cette capacité de rétention rend le LSTM idéal pour la génération de musique. Une partition n'est pas une suite de notes aléatoires, mais un ensemble dynamique doté d'une continuité narrative. Pour composer un morceau cohérent, le modèle doit maintenir le fil conducteur, tel que la tonalité, le tempo ou le thème principal, introduit de nombreuses mesures en amont. En traitant les données pas-à-pas tout en préservant cet historique, le LSTM assimile la grammaire musicale et structure les mélodies de manière logique du début à la fin.

- **L'Architecture** : L'encodeur devient un LSTM bidirectionnel (qui lit la musique de gauche à droite et inversement pour capter un contexte global) et le décodeur devient un LSTM autorégressif (générant la séquence note par note).
- **Le Teacher Forcing avec Décalage (Shift)** : Un défi majeur de l'apprentissage des séquences est d'empêcher le modèle de tricher. Si l'on fournit au décodeur le token au temps  $t$  pour prédire le token au temps  $t+1$ , le réseau peut apprendre une règle trop simple consistant à associer directement chaque entrée à la sortie attendue, sans réellement apprendre les dépendances et la structure de la séquence. L'erreur d'apprentissage devient alors très faible, mais le modèle reste incapable de générer correctement une nouvelle séquence de manière autonome. La solution est le "Teacher Forcing avec décalage" : pour prédire le token au temps  $t+1$ , on ne donne au modèle que l'historique strict (de  $t=0$  à  $t1$ ). Le réseau est ainsi contraint d'apprendre la véritable grammaire musicale REMI pour deviner la suite.

- **Résultats** : Les courbes de train et de validation sont quasi confondues, il n’y a pas de surapprentissage. La perte totale lors de la validation atteint près de 1300 contre 2100 pour le CNN et 2400 pour le MLP. On observe ainsi une nette amélioration. L’élément flagrant et la structuration de l’espace latent, ici contrairement au MLP ou au CNN qui montraient un nuage dense, le LSTM produit des petits amas isolés mais totalement multicolores. Chaque micro-groupe contient des points de toutes les classes (Q1, Q2, Q3, Q4). Ici, le fait que les amas soient isolés montre que l’encodeur essaie encore de placer les données à des coordonnées précises, mais le fait qu’ils soient multicolores prouve que ces coordonnées n’ont aucun rapport avec l’émotion, il projette les données au hasard ou selon un critère qui n’a rien à voir avec tes classes. En somme, le modèle ne comprend plus l’émotion demandée via l’espace latent, mais il réactive des motifs cohérents (lent/triste ou rapide/joyeux) qu’il a appris à générer de manière autonome.
- **Bilan musical** : On observe une nette amélioration par rapport au modèle CNN : les musiques générées sont plus riches, plus cohérentes et globalement mieux structurées dans le temps. Le modèle parvient également à capturer certaines caractéristiques émotionnelles de la musique. Par exemple, dans la génération Q1, les notes s’enchaînent rapidement avec une dynamique plus énergique et une présence plus marquée d’accords majeurs, ce qui correspond à une ambiance joyeuse. À l’inverse, la génération Q2, censée exprimer une émotion plus triste, présente un tempo plus lent et une proportion plus importante d’accords mineurs, ce qui montre que le modèle a appris des corrélations pertinentes entre structure musicale et émotion. Cependant, cette approche reste limitée par le fait que l’espace latent du VAE est non structuré de manière explicite vis-à-vis des attributs musicaux (émotion, tempo, tonalité). Autrement dit, ces caractéristiques sont bien apprises, mais de façon implicite et entremêlée, ce qui rend le contrôle difficile et peu fiable : on ne peut pas garantir qu’une direction précise dans l’espace latent corresponde toujours à une émotion ou un paramètre musical donné.

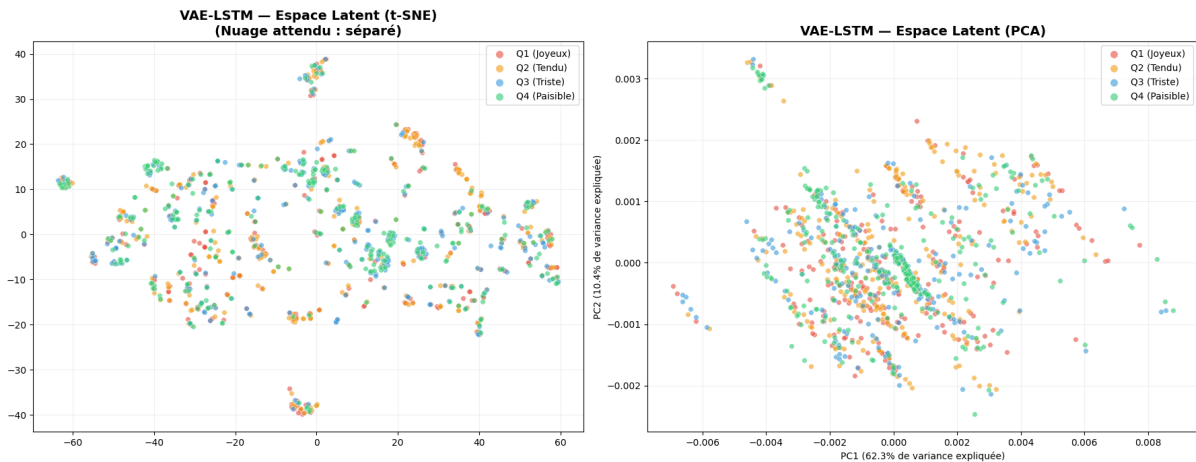


FIGURE 7 – t-SNE du VAE LSTM (dim =64)

Malgré la puissance du modèle VAE-LSTM, une ultime limite demeurerait : la génération était non-dirigée. Il restait impossible de dicter une intention émotionnelle spécifique (choisir le quadrant de Russell), justifiant notre transition finale vers un modèle conditionné : le C-VAE.

## 4 Le C-VAE : Auto-Encodeur Variationnel conditionnel

Bien que la combinaison d’un Auto-Encodeur Variationnel et de réseaux récurrents (LSTM) ait permis de générer des séquences musicales assez réalistes et fluides, le modèle ne permettait pas de répondre à notre objectif initial : la génération de musique en suivant les émotions. Les résultats obtenus avec le VAE restait aléatoire. En échantillonnant un vecteur  $z$  au hasard, le décodeur produisait une mélodie, mais il nous était impossible de dicter l’émotion de cette mélodie (Joie, Tristesse, Colère, Calme). Pour pallier ce problème, nous avons fait évoluer notre architecture vers un Auto-Encodeur Variationnel Conditionnel (C-VAE).

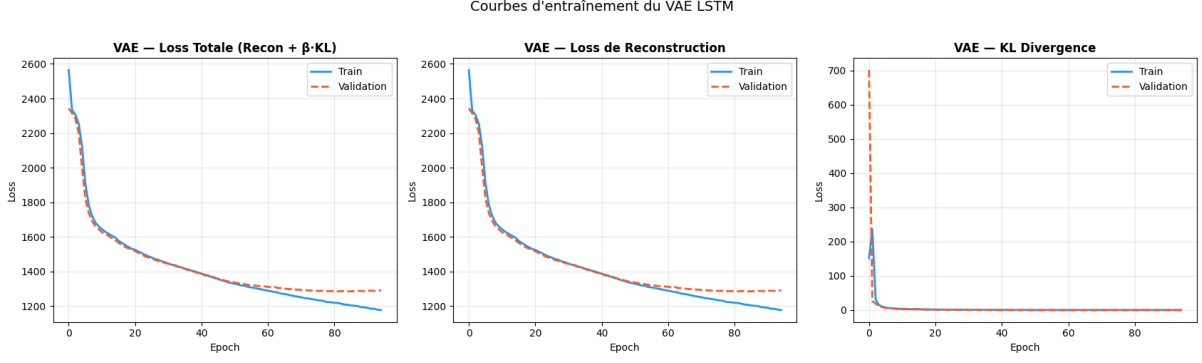


FIGURE 8 – Courbe de Loss du VAE LSTM (dim =64)

#### 4.1 L'architecture d'un C-VAE

Le C-VAE (*Conditional Variational Autoencoder*) est une extension directe du VAE.<sup>7</sup> La différence fondamentale réside dans l'introduction d'une information supplémentaire, appelée la **condition** (notée  $c$ ).

Dans notre projet, cette condition  $c$  représente le label émotionnel du morceau (Q1, Q2, Q3 ou Q4) encodé sous forme de vecteur avec la méthode du *One-Hot encoding*. (on a obtenu un vecteur binaire de dimension 4, par exemple  $[1, 0, 0, 0]$  pour indiquer explicitement le quadrant Q1).

Mathématiquement, les distributions d'un VAE classique dépendent uniquement des données  $x$  et de l'espace latent  $z$  :<sup>8</sup>

$$\text{Encodeur} : q(z|x) \quad \text{et} \quad \text{Décodeur} : p(x|z)$$

Dans un C-VAE, ces distributions sont toutes deux conditionnées explicitement par le vecteur  $c$  :

$$\text{Encodeur} : q(z|x, c) \quad \text{et} \quad \text{Décodeur} : p(x|z, c)$$

Le processus global devient alors :

$$x, c \rightarrow \text{Encodeur} \rightarrow \mu, \log(\sigma^2) \rightarrow z \sim \mathcal{N}(\mu, \sigma^2) \rightarrow [z, c] \rightarrow \text{Décodeur} \rightarrow \hat{x}$$

L'intégration de cette condition transforme radicalement le comportement de l'encodeur et du décodeur :

##### 1. Au niveau de l'Encodeur :

L'encodeur LSTM ne reçoit plus uniquement la séquence musicale  $x$ . À chaque pas de temps, nous concaténons le vecteur conditionnel  $c$  aux embeddings de la musique. L'encodeur sachant déjà à quelle émotion correspond le morceau (grâce à  $c$ ), il n'a plus besoin de stocker l'information émotionnelle dans l'espace latent  $z$ . Le vecteur  $z$  se spécialise uniquement dans la capture de la structure rythmique et du style propre au morceau.

##### 2. Au niveau du Décodeur :

Le décodeur fait de la génération conditionnée. Pour composer un morceau sur commande, nous tirons un vecteur de style  $z$  aléatoirement depuis notre distribution Gaussienne  $\mathcal{N}(0, 1)$ , puis nous y concaténons l'émotion désirée  $c$ . Le décodeur LSTM est initialisé avec ce vecteur combiné  $[z, c]$ . À chaque note qu'il génère, la condition  $c$  lui rappelle le quadrant visé, forçant le modèle à adapter la vitesse, le tempo et le registre harmonique pour coller à l'émotion demandée.

## 5 Génération et Résultats :

Une fois le C-VAE entraîné, la phase de génération (l'inférence) ne consiste pas simplement à prendre la prédiction mathématique brute la plus probable. La musique requiert de la créativité et de la variance. Si l'on choisit toujours le token ayant la probabilité maximale, le résultat ne sera pas aussi satisfaisant et le modèle peut être répétitif. À l'inverse, si l'on tire complètement au hasard selon les probabilités, le

modèle risque de générer des notes hors gamme (dissonances). Pour garantir une composition à la fois originale et harmonieuse, nous avons joué avec La Température : La température modifie la distribution des probabilités avant l'échantillonnage. Une température de 0 ne change rien. Une température très élevée "écrase" les probabilités (tout devient équiprobable, générant du chaos). Après plusieurs tests, nous avons trouvé qu'une température basse de 0.6 était le meilleur compromis : le modèle prend moins de risques et favorise les notes structurellement logiques, ce qui est crucial pour maintenir la cohérence harmonique de la musique classique et pop.

## 5.1 Analyse de la Convergence et de la Séparation Émotionnelle

L'analyse des graphiques générés à l'issue de l'entraînement du C-VAE valide pleinement notre architecture. Afin d'étudier l'impact de la compression, nous avons comparé deux modèles avec des tailles d'espace latent différentes : une dimension de 32 et une dimension de 64.

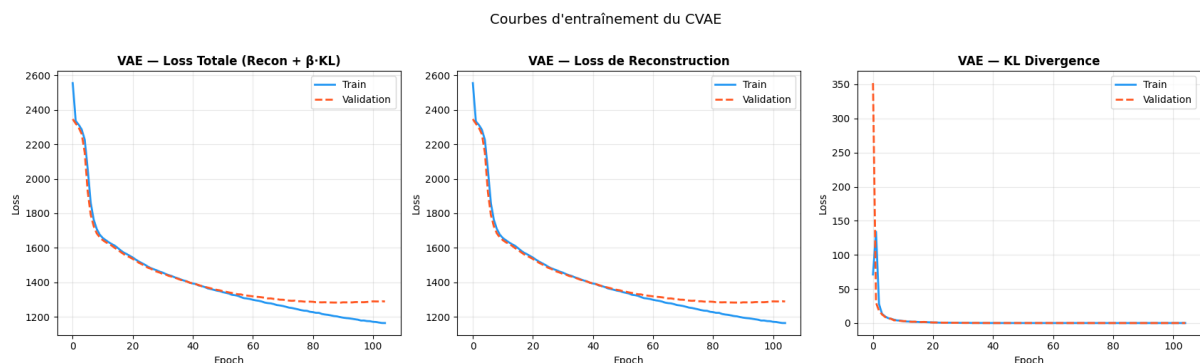


FIGURE 9 – Courbes de loss du CVAE LSTM (dim = 32)

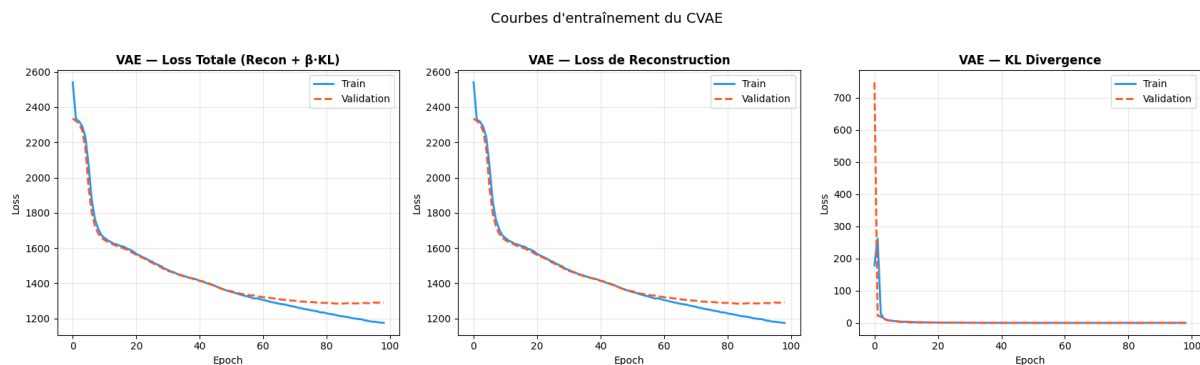


FIGURE 10 – Courbes de loss du CVAE LSTM (dim = 64)

### 1. Analyse des Courbes de Loss :

L'observation des courbes d'entraînement démontre un apprentissage très stable dans les deux cas. La courbe de **Loss Totale** ainsi que la courbe de **Reconstruction** montrent une descente fluide et une superposition quasi-parfaite entre le jeu d'entraînement et le jeu de validation (la perte chutant de 2600 à environ 1180). Cela prouve l'absence de surapprentissage (*overfitting*) dans notre modèle. La différence majeure entre les 2 dimensions d'espace latent réside dans la **Divergence KL** : le pic initial s'élève à environ 350 pour  $d = 32$ , mais culmine à près de 750 pour  $d = 64$ . C'est mathématiquement logique : l'espace latent étant deux fois plus grand, la pénalité imposée à l'encodeur par rapport à la Gaussienne standard est mécaniquement plus forte au début de l'apprentissage.

### 2. L'évolution de l'espace latent (PCA et t-SNE) :

Nous avons vu que sur les données brutes et le VAE la projection t-SNE ne montrait qu'un immense nuage chaotique où les quatre émotions se superposaient. Après l'entraînement du C-VAE, on peut bien distinguer les différents clusters représentant les 4 émotions dans l'espace latent. Nous observons des différences entre les 2 dimensions :

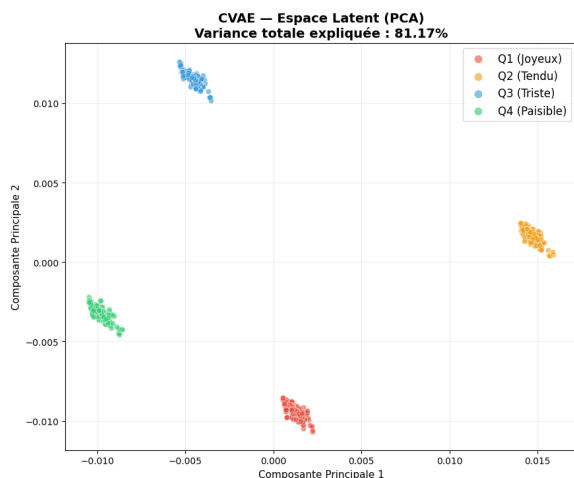


FIGURE 11 – PCA (Dimension 32)

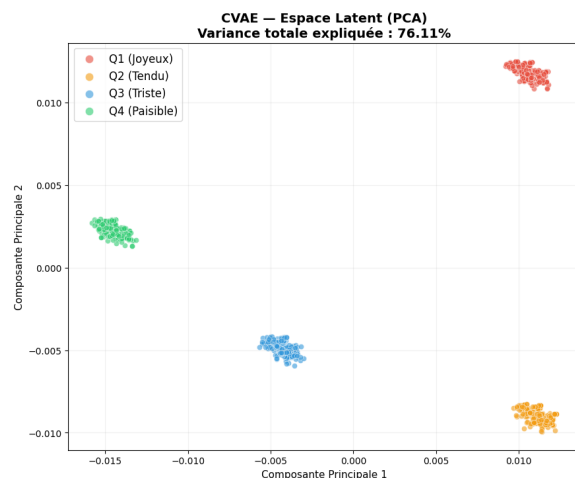


FIGURE 12 – PCA (Dimension 64)

- **Sur la PCA (Variance) :** L'algorithme linéaire parvient à expliquer **81.17%** de la variance totale pour  $d = 32$ , contre **76.11%** pour  $d = 64$ . Cette légère baisse peut s'expliquer par l'augmentation de la difficulté de projeter linéairement sur 2 axes la variance d'un espace à 64 dimensions contrairement à celle d'un espace à 32 dimensions. Dans les deux cas, la PCA trace quatre groupes distincts correspondant exactement aux quatre quadrants de Russell (Q1 Joyeux en rouge, Q2 Tendu en orange, Q3 Triste en bleu, Q4 Paisible en vert).

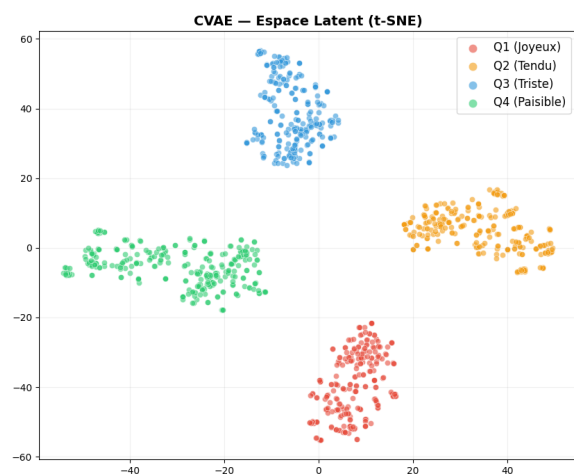


FIGURE 13 – t-SNE (Dimension 32)

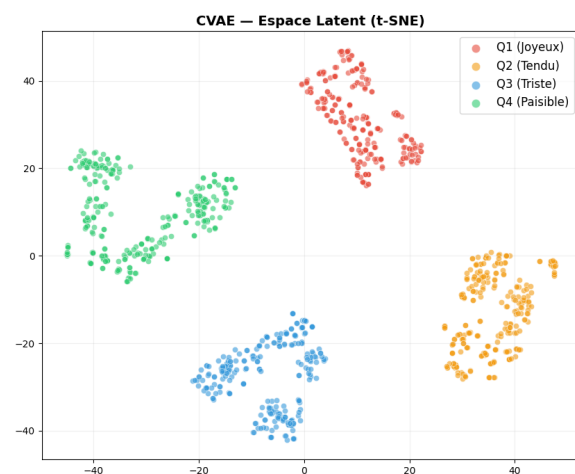


FIGURE 14 – t-SNE (Dimension 64)

- **Sur le t-SNE (Topologie) :** Comme pour la PCA, l'espace latent est parfaitement scindé en quatre "clusters" isolés correspondant aux quatre quadrants de Russell. Cependant, on observe que pour  $d = 32$ , les clusters sont très denses et resserrés. À l'inverse, pour  $d = 64$ , les clusters sont beaucoup plus "étalés" et dispersés. Cet étalement prouve qu'un espace latent plus grand donne au modèle plus de "place" (de degrés de liberté) pour encoder des variations stylistiques subtiles à l'intérieur d'une même émotion, là où  $d = 32$  force une compression plus stricte et brutale. Cette séparation mathématique absolue prouve que le modèle a véritablement réussi à extraire, comprendre et regrouper les signatures structurelles de chaque émotion.

### 3. L'évaluation qualitative (L'écoute) :

Lors de la génération de nouveaux morceaux en forçant le vecteur de condition  $c$ , le modèle produit des résultats distincts qui reflètent plutôt bien l'émotion souhaitée. Une commande Q1 ou Q2 force le décodeur à générer des séquences rapides avec une forte vélocité. À l'inverse, une commande Q4 force un tempo apaisé. Il est toutefois toujours difficile d'avoir à l'écoute un avis unanime sur l'émotion transmise, cela relevant de la subjectivité de chacun. On peut néanmoins souligner l'amélioration de la qualité musicale du son généré pour  $d = 64$  par rapport à  $d = 32$ . Cette observation auditive corrobore directement notre analyse du graphique t-SNE : l'espace latent plus vaste de  $d = 64$  offre au décodeur une plus grande richesse de nuances stylistiques et harmoniques pour composer, rendant la mélodie moins "compressée" et plus naturelle.

## 6 Conclusion et Perspectives

Malgré la réussite de notre architecture finale (CVAE-LSTM) qui génère des textures émotionnelles, une limite majeure persiste : la cohérence musicale à long terme. Un LSTM peine à maintenir une macro-structure compositionnelle sur plusieurs minutes, comme la répétition d'un thème ou d'un refrain.

### 6.1 Bilan des Modèles

Le tableau ci-dessous récapitule les performances finales obtenues pour chaque architecture testée lors de la phase de validation :

| Modèle       | Dimension Latente | Loss Finale (Val) | Observation Principale                                              |
|--------------|-------------------|-------------------|---------------------------------------------------------------------|
| VAE (MLP)    | 64                | 2346.0            | Surapprentissage massif, KL Collapse                                |
| VAE (CNN)    | 64                | 1885.4            | Convergence stable mais amnésie temporelle et pause entre les notes |
| VAE (LSTM)   | 64                | 1284.4            | Bonne grammaire génération d'émotions aléatoire                     |
| C-VAE (LSTM) | 64                | 1283.6            | Meilleure qualité audio, clusters étendus                           |

TABLE 2 – Comparaison des performances de validation selon les architectures.

#### Analyse des performances :

L'analyse de l'évolution de la perte de validation justifie l'évolution de nos modèles. L'échec du MLP se traduit par une perte extrêmement haute (2346.0). Le passage au CNN montre une première amélioration significative (1885.4) en capturant des motifs locaux, mais reste limité. La véritable rupture se produit avec l'introduction des réseaux récurrents LSTM : la perte s'effondre à 1284.4. Cette chute drastique prouve que la modélisation de la musique est un problème séquentiel nécessitant la mémorisation de l'historique des tokens. Enfin, le modèle conditionné final (CVAE) obtient un score quasi identique (1283.6). Cette stagnation apparente de la perte cache en réalité une prouesse : le modèle parvient à maintenir le même niveau d'excellence grammaticale tout en contraignant l'espace latent pour y injecter un contrôle émotionnel parfait.

Pour améliorer notre modèle, d'autres pistes peuvent être testées, la littérature scientifique sur le Deep Learning appliqué à la musique<sup>6</sup> présente d'autres méthodes pour générer de la musique. Les mécanismes d'attention au cœur des *Transformers* résolvent le problème de l'amnésie des LSTM en reliant mathématiquement une note à un accord joué plusieurs centaines de pas temporels auparavant. Enfin, comme souligné dans le cours de G. Richard,<sup>6</sup> l'utilisation des réseaux antagonistes génératifs (GAN) représente une alternative puissante aux VAE. En utilisant un réseau "Discriminateur" chargé de différencier la vraie musique humaine de la musique générée, le modèle est forcé de produire des morceaux beaucoup plus réalistes sur le plan stylistique.

## Références

- [1] Hung, H.-T., Ching, J., Doh, S., Kim, N., Nam, J., & Yang, Y.-H. (2021). *EMOPIA : A Multi-Modal Pop Piano Dataset For Emotion Recognition and Emotion-based Music Generation*. Dans *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*.
- [2] Fradet, N., Briot, J.-P., Chhel, F., El Fallah-Seghrouchni, A., & Gutowski, N. (2021). *MidiTok : A Python Package for MIDI File Tokenization*. HAL open archive (hal-03418930).
- [3] Huang, Y.-S., & Yang, Y.-H. (2020). *Pop Music Transformer : Beat-based Modeling and Generation of Expressive Pop Piano Compositions*. Dans *Proceedings of the 28th ACM International Conference on Multimedia* (pp. 4200-4208).
- [4] van der Maaten, L., & Hinton, G. (2008). *Visualizing Data using t-SNE*. *Journal of Machine Learning Research*, 9, 2579-2605.
- [5] Shlens, J. (2014). *A Tutorial on Principal Component Analysis*. arXiv preprint arXiv :1404.1100.
- [6] Richard, G., & Essid, S. (2021). *Deep Learning for Audio and Music*. Support de cours (MVA), Télécom Paris, Institut Polytechnique de Paris.
- [7] Sohn, K., Lee, H., & Yan, X. (2015). *Learning Structured Output Representation using Deep Conditional Generative Models*. Dans *Advances in Neural Information Processing Systems (NeurIPS)*, 28.
- [8] Vérine, A. *Generative AI (GenAI) - Course Slides*. Support de cours, LAMSADE, Université Paris Dauphine-PSL.